

PSA Diag

Build your own diagnostic cable

Arduino or ESP32 + MCP2515 CAN interface for Peugeot / Citroen / DS / Opel

Manual - version 1.0 - 26 July 2026

Based on the `arduino-psa-diag` firmware (ludwig-v, GPLv3). For vehicles you own or are authorised to service only. Use at your own risk.

1. What you are building

This cable is a USB-to-CAN bridge: a microcontroller (Arduino or ESP32) with an MCP2515 CAN controller talks to the car's diagnostic CAN bus through the OBD2 connector, and to the PSA Diag app over USB serial. The app sends UDS/KWP commands; the microcontroller puts them on the CAN bus and returns the replies.

The firmware (arduino-psa-diag) is fixed; which ECU you address is decided by the app (e.g. the NAC on 764:664). So you build the hardware once and use it for every ECU.

! Safety and scope

Use this only on vehicles you own or are authorised to service. Work with the ignition off unless a step says otherwise. The CAN bus is a 12V environment: never short CAN-H/CAN-L to +12V or ground. Reading is safe; writing/coding can misconfigure an ECU - do that at your own risk and with a backup.

2. Bill of materials

Part	Detail / choice	Qty
Microcontroller	Arduino Uno or Nano (5V) OR an ESP32 dev board	1
CAN module	MCP2515 + TJA1050, with a 16 MHz crystal (mind the frequency!)	1
OBD2 cable	OBD2 extension cable (male-female) to tap into, or a bare OBD2 male plug	1
Wires	Dupont jumper wires	~8
USB cable	To match your board (USB-B / micro / USB-C)	1
Optional	120 ohm resistor (only if the bus is not already terminated)	0-1

Mind the MCP2515 crystal frequency: many modules use 8 MHz, some 16 MHz. This MUST match the firmware setting (chapter 6), otherwise CAN communication will not work.

3. Wiring - Arduino Uno / Nano (standard Arduino)

3a. MCP2515 to Arduino (SPI)

MCP2515 pin	Arduino pin	Function
VCC	5V	Module power
GND	GND	Ground
CS	D10	SPI chip-select (CS_PIN_CAN0 = 10)
SI (MOSI)	D11	SPI data to module
SO (MISO)	D12	SPI data from module
SCK	D13	SPI clock
INT	D2	Interrupt (recommended)

3b. MCP2515 (CAN side) to the OBD2 connector

MCP2515 CAN	OBD2 pin	Note
CANH	6	CAN-High (HS-CAN diagnostics)
CANL	14	CAN-Low (HS-CAN diagnostics)
GND	4 and/or 5	Chassis / signal ground

Power: the Arduino can be powered over USB from your laptop. To power it from the car, OBD2 pin 16 = +12V battery (via a proper 12V->5V regulator, NOT straight onto 5V).

4. Wiring - ESP32 + MCP2515

The ESP32 uses the same MCP2515 module but different SPI pins. The firmware is the same; only change CS_PIN_CAN0 to the chosen CS pin (chapter 6).

MCP2515 pin	ESP32 pin (VSPI)	Function
VCC	5V (VIN)	Power module at 5V
GND	GND	Ground
CS	GPIO 5	SPI chip-select
SI (MOSI)	GPIO 23	SPI data to module
SO (MISO)	GPIO 19	SPI data from module
SCK	GPIO 18	SPI clock
INT	GPIO 4	Interrupt

! 3.3V vs 5V logic

The ESP32 is 3.3V; the MCP2515 module is 5V. The MISO line (SO -> GPIO19) arrives at the ESP32 at 5V. Many modules work directly, but a level shifter or a 3.3V-capable MCP2515 is the clean/safe choice. Do power the module at 5V (VIN), otherwise the CAN transceiver is unreliable.

The CAN side to OBD2 is identical to chapter 3b (CANH->6, CANL->14, GND->4/5).

5. OBD2 connector pinout (diagnostic CAN)

OBD2 pin	Signal	To
6	CAN-High	MCP2515 CANH
14	CAN-Low	MCP2515 CANL
4	Chassis ground	MCP2515 GND
5	Signal ground	MCP2515 GND (may replace 4)
16	+12V battery	Only if powering from the car (via a regulator)

Tip: use an OBD2 extension cable and tap CAN-H (6), CAN-L (14) and ground (4), so the original socket stays intact and you can keep the Diagbox/tester alongside.

6. Firmware and source code

The bridge firmware is open source (GPLv3). Download the sketch and the two libraries, then flash them onto your board. Nothing here is secret - the source is public. Get it from our repository:

Component	Source (GitHub)	License
Firmware (our repo)	github.com/sabsteef/arduino-psa-diag	GPLv3
- original / upstream	github.com/ludwig-v/arduino-psa-diag	GPLv3
MCP2515 library	github.com/autowp/arduino-mcp2515	MIT
Thread library	github.com/ivanseidel/ArduinoThread	GPLv3
Seed-key algorithm	github.com/ludwig-v/psa-seedkey-algorithm	GPLv3

The sketch is `arduino-psa-diag/arduino-psa-diag.ino` in our repository. It is a GPLv3 fork of the original by Ludwig V. (credited above); all copyright notices are preserved. Take it from the repo rather than a copy so you always have the exact, unmodified source and the full licence text.

Quick Arduino config (copy/paste settings + wiring), online in the same repo:

```
github.com/sabsteef/arduino-psa-diag/blob/master/ARDUINO-CONFIG.md
```

7. Flashing the firmware

- Install the Arduino IDE (or PlatformIO). For ESP32: add the ESP32 board package via Boards Manager.
- Install the libraries: 'autowp MCP2515' (arduino-mcp2515) and 'ArduinoThread'.
- Open arduino-psa-diag.ino from the repository in chapter 6.
- Select your board (Arduino Uno/Nano or your ESP32 variant) and the correct serial port.
- Check the configuration below, then click Upload.

7a. Configuration in the firmware (top of the .ino)

```
#define CS_PIN_CAN0 10           // Arduino: 10 | ESP32: 5
#define SERIAL_SPEED 115200     // fixed - do not change
#define CAN_SPEED CAN_500KBPS   // diagnostic CAN = 500 kbps
#define CAN_FREQ MCP_16MHZ      // 16 MHz crystal;
                                // set to MCP_8MHZ for an 8 MHz module
```

! Most common mistake

Wrong CAN_FREQ (8 vs 16 MHz) or wrong CS pin = no communication (error 7F3E03 / 'connection lost').
Check the crystal on your MCP2515 module and set CAN_FREQ to match.

8. App-side configuration

The PSA Diag app talks at 115200 baud, 8N1, with LF (newline) only as the line ending. The ECU choice (CAN addresses, unlock key) lives in the app data, not the firmware - you pick the ECU in the interface.

8a. Pick the port

```
# Desktop/CLI - find the port and test:
npm run diag -- ports
npm run diag -- version --port /dev/tty.usbserial-XXXX
npm run diag -- info nac --port /dev/tty.usbserial-XXXX

# or fix the port so --port is not needed:
export PSA_PORT=/dev/tty.usbserial-XXXX
```

In the web app (browser) you pick the port via the Web Serial port chooser; the settings (115200, LF) are already correct.

8b. Configuration summary

Setting	Value	Where
Serial	115200 baud, 8N1, LF	firmware + app (fixed)
CAN speed	500 kbps (HS-CAN)	firmware CAN_SPEED
Crystal	16 MHz (or 8 MHz)	firmware CAN_FREQ
CS pin	Arduino 10 / ESP32 5	firmware CS_PIN_CAN0
ECU choice	in the app	PSA Diag interface

9. First test and troubleshooting

- Plug the cable into OBD2, connect USB to the laptop, ignition on.
- Open the app or CLI, pick the port, request 'version' and 'info nac'.
- Reading works? Then the wiring is good. Only then move on to writing/coding (with a backup).

Symptom	Likely cause / fix
7F3E03 / connection lost	Wrong CAN_FREQ (8/16 MHz), wrong CS pin, CAN-H/L swapped, or no 120 ohm termination on the bus.
No serial port	USB cable is 'charge only', driver missing (CH340/CP2102), or wrong port selected.
Connected but no reply	Ignition off, wrong ECU selected, or CAN-H/CAN-L reversed (pin 6 <-> 14).
Random errors	Bad Dupont connection, module powered at 3.3V instead of 5V, or ground (pin 4/5) not connected.

! Finally

This manual covers building the interface. Diagnostics are safe; coding and firmware/CAL flashing can render an ECU unusable if done wrong or interrupted. Always back up the zone/config first and work on a vehicle you are allowed to service.